Experimento A · Hallazgos y aprendizaje

Ío en Minecraft: un modelo propio, pequeño y local que decide la movilidad y las acciones del cuerpo · 13-14 de octubre de 2026 · actualizado tras las fases C y D

1. Qué se construyó

- **Cuerpo:** bot Mineflayer con puerto de control local; ahora una clase *Personaje* admite varios a la vez (Io, Eco, Iris), cada uno con su puerto, memoria y perfil.
- **Cerebro:** red neuronal pequeña (archivo de ~100 KB) entrenada en el propio ordenador, sin claves de API ni Colab. Primero elige pasos (avanzar, girar, picar) con 10 entradas; después elige entre 5 habilidades con 20-25 entradas.
- **Habilidades** (código determinista con resultado y causa): ir a un árbol, picar el tronco, recoger, ir hacia un jugador, huir.
- **Perfiles** (recolector, constructor, cooperativo, supervivencia, explorador): 4 pesos que entran a la red como entradas.
- **Zonas protegidas** definidas por el usuario (*zonas.json*): los personajes no talan ni, si son vedadas, pisan esas zonas. Se marcan desde el chat del juego (!zona inicio / fin NOMBRE).

2. Cifras clave

Hito	Resultado	Qué cambió
Punto de partida (banco de 30)	15/30 · 50 %	Política con 3 entradas
Visión lateral (10 entradas)	18/30 · 60 %	Obstáculos a izquierda y derecha
Banco corregido	22/30 · 73 %	Sin caer en copas ni en el mar; sin picar al aire
Primer banco de 60	23/60 · 38 %	Falso: el banco había talado el arbolado
Banco de 60 con troncos repuestos	47/60 · 78 % ±10	Cifra fiable
Elegir árbol por camino real	52/60 · 87 %	Descarta árboles al otro lado del agua
Confirmación (otra semilla)	46/60 · 77 %	Conjunto: 98/120 = 82 %
Vallas y escalones	55/60 · 92 %	Salto manual corriendo; repetido con igual resultado
Nueva línea base (zona inundada excluida)	54/60 · 90 %	Con 25 entradas y perfiles
Tres personajes a la vez	159/180 · 88 %	Io 52, Eco 54, Iris 53 de 60; uno solo: 54/60

Habilidades (fase 2)

- ir a un árbol 26/30; picar 52/52; huir 8/8; ir hacia un jugador 8/8; recoger de 8/14 a un 92 % tras corregirla (se perdía persiguiendo huevos tras una valla).
- Red elige habilidades vs guion escrito a mano, en Minecraft: 61,8 ± 5,4 frente a 60,5 ± 6,2 troncos cada 5 minutos: empate. En el simulador: guion 16,8, azar 9,1, DQN solo 14,5-15,8, DQN con imitación 16,7-17,0.
- Perfiles (banco real, aldeano en un claro): cooperativo 3,2 troncos y 64 % del tiempo junto al jugador; recolector 51,2 troncos y 0 %.
- Coste de varios personajes: 20-60 % de un núcleo y 200-300 MB cada uno.

Varios personajes (fases C y D)

- **Simulador con N personajes** que comparten una sola red y una sola memoria; cada uno ve la posición del compañero y su árbol objetivo (25 entradas) y dispone de una habilidad nueva, ir a otro árbol. Con dos recolectores, los conflictos bajan de 10,9 a 1,5 por partida (y de 22,4 a 4,7 con tres).
- **Minecraft, tres a la vez:** 159/180 (88 %) frente a 90 % de uno solo. Fuera de las zonas protegidas, 39/40 cada uno. Pero al empezar los tres delante del mismo árbol, en 39 de 54 puntos picaron el mismo bloque.
- **Madera del equipo:** con el guion, tres personajes sacan 54-90 troncos en 4 minutos frente a 47-55 de uno solo: la ganancia coincide con los bloques distintos que se pican; entre 36 y 54 bloques por episodio los picaron dos o tres a la vez. Arreglo ya hecho en el bot (no pica el bloque de un compañero), pendiente de medir.
- **Carga del servidor:** 36 % de un núcleo y 16 ms por tick sin bots; con tres trabajando, 46-51 % y 18 ms de media (máximo 23 ms; el límite son 50). Los bots, 55-63 % de un núcleo y 430-520 MB en total.
- **Zonas protegidas:** con las tres zonas del usuario, 20 de los 60 puntos del banco quedan fuera; en los 40 restantes, Io solo acierta 39/40.
- **Red nueva (validación en simulador, 40 mundos):** recolector 16,6 troncos; cooperativo 7 con el 60 % del tiempo junto al jugador; dos recolectores, 20,4-22,6 cada uno con 1-1,5 conflictos; tres, 20,3-22,6 con 2,4-3. Aún no medida en Minecraft.

3. Lo que aprendimos

> **1. Medir bien cuesta tanto como entrenar bien.** Con 30 pruebas el ruido es de ±3 aciertos: casi todas las «mejoras» eran suerte. Con 60 pruebas el margen baja a ±10 puntos y ya se pueden comparar variantes.

> **2. El banco de pruebas también puede equivocarse.** Falseaba las cifras de tres formas: dejaba a Ío sobre las copas, lo hacía aparecer bajo el mar, y cada acierto talaba un árbol real hasta agotar el arbolado (38 % falso). Hay que reponer el escenario después de cada prueba.

> **3. Lo que la red no puede ver se arregla fuera de la red.** Árboles inalcanzables, vallas de metro y medio o escalones se resuelven en la elección del objetivo y en la ejecución, no entrenando más.

> **4. La brecha simulador-mundo real aparece siempre.** El simulador es más limpio (siempre hay camino al árbol). Cada fallo en Minecraft debe trasladarse al simulador; en cambio, los fallos de ejecución (navegador atascado) se arreglan en el bot.

> **5. Con pocas señales, el guion ya es casi óptimo.** La red sola quedaba por debajo; igualó al guion solo aprendiendo de él (DQN con imitación). Para aportar algo más hacen falta perfiles, señales nuevas o habilidades nuevas.

> **6. Las observaciones del usuario valen oro.** El pantano junto al mar y la valla con alfombra que un jugador salta y el bot no fueron hallazgos suyos, imposibles de ver en las cifras.

> **7. Compartir un modelo entre varios personajes funciona** si cada uno recibe su perfil y la posición del compañero; con tres personajes el rendimiento individual apenas cae (88 %

frente a 90 %).

8. Compartir red no basta: hay que repartir el trabajo. Tres personajes con la misma red eligen el mismo árbol y se estorban. Hace falta una regla explícita de ceder el árbol y que el simulador la haga necesaria (posiciones distintas, conflictos medidos).

9. Un banco que modifica el mundo debe poder deshacerse. Cortar una serie a medias dejó 74 troncos talados dentro de una zona protegida del usuario y objetos apartados sin devolver. Conviene un script de reposición y registrar siempre lo que se aparta.

10. Antes de dar por buena una cifra, mira de dónde sale. Tres de los fallos de mayor tamaño de esta sesión (copas, mar y arbolado agotado) eran del banco, no del modelo.

4. Pendiente

- La orden *soltar* del bot no tira nada (tossStack no tiene efecto); se resolvió con un comando del servidor.
- Elegir la política del entrenamiento con varios agentes (*politica_habilidades_equipo_s0/s1.json*), copiarla a *politica_habilidades.json* y medir la red en Minecraft: sola, en dúo y en trío, y con perfiles opuestos a la vez.
- Rehacer la línea base de *medir.py* con las zonas excluidas (los puntos se redibujan) y medir tres personajes con el arreglo de picar entre compañeros.
- Regla de ceder el árbol cuando dos personajes están delante del mismo; calibrar con lo medido el supuesto de «estorbo» del simulador (hoy 0,75).
- Los objetos apartados del inventario de Io (384 de roble, 192 de roble oscuro, 229 huevos), Eco (9+9) e Iris (5+3) siguen guardados; hay que devolverlos o decidir qué hacer con ellos.
- Constructor y explorador se comportan como el recolector: faltan habilidades propias.
- Probar la huida con hostiles reales (el servidor está en pacífico) y la habilidad de dormir.
- El entrenamiento con varios agentes (5000 episodios, dos semillas) sigue en segundo plano en el ordenador.

5. Cómo retomar

- Servidor: *servidor/arrancar.sh*. Bots: *node io.js Io Eco:3002:recolector Iris:3003:recolector* en *bot/* (parar con *fuser -k PUERTO/tcp*).
- Banco: *entrenamiento/medir.py 60 42* (repone troncos; descarta puntos dentro de *zonas.json*). Ejecutor: *entrenamiento/medir_ejecutor.py* (opción *--equipo* para varios personajes) y *medir_carga.py* para la carga.
- Informe completo y pasos para retomar, uno a uno, en el archivo del trabajador (fases C y D); reposición de troncos con *reponer_serie.py* (sin *--hacer* solo lista).
- Todo el detalle, ciclo a ciclo, está en *mente/diario.md*; el plan, en *PLAN.md*.

Los entrenamientos se hacen en local y no gastan tokens; sí los gasta el trabajo de programarlos y medirlos. Cifras tomadas del diario del experimento; los márgenes son intervalos del 95 %.